

Prompt: build the "AEO Crawler Readiness" skill

Paste the block below to Claude with the **skill-creator** skill enabled. It builds a reusable skill that runs the same diagnosis-and-fix workflow we ran on job-genie.ai/blog, generalized to any content site.

Use the skill-creator skill to build a new skill.

Skill name: `aeo-crawler-audit` **One-line purpose:** Audit a content site's visibility to AI answer engines and search crawlers, diagnose why pages aren't being cited/indexed, and produce an ordered remediation plan with ready-to-ship fixes.

Triggering description (write it so the skill fires on all of these): any request to assess AEO/GEO readiness, "how does my blog rank for AI crawlers," AI-answer-engine or LLM citability, indexation problems, `site:` coverage gaps, canonical/host splits, domain masking or forwarding issues, keyword cannibalization / duplicate posts, sitemap/robots problems, or "why isn't my content getting indexed/cited." Trigger on a bare blog or site URL plus any of those intents. Do NOT trigger for general copywriting, keyword research, or paid-ads work.

Inputs the skill should collect (ask only if not provided):

- Target URL (blog index or site root) and the canonical host preference (default: `www`), to match existing canonical tags).
- Hosting + registrar (default assumption: Replit deployment behind a GoDaddy domain) and app framework (Express / Next.js / static) — needed to emit the right redirect code.
- Whether Google Search Console access exists (affects the re-index step).

Workflow — run in this order, with an approval gate after Phase 1 before proposing fixes:

Phase 1 — Discoverability diagnosis (read-only).

1. Fetch the target URL. Capture: `canonical`, `robots` meta, OG/Twitter tags, publish dates, and the host used in internal links.
2. Test host consistency: does the bare domain AND the `www` host each return `200`? Is there a real `301` between them? Do internal links match the declared canonical host? Flag any contradiction (e.g., bare-host pages linking to bare URLs while canonical says `www`).
3. Detect masking: apex returning `200` with a frame wrapper instead of a `301` = GoDaddy-style forwarding/masking. Call it out explicitly.
4. Estimate indexation: run `site:domain`, count retrievable pages, and compare against the number of posts on the index. A large gap = discoverability failure.

5. Verify `sitemap.xml` exists, is referenced in `robots.txt`, lists every post on the canonical host, and is submitted in GSC.
6. Output a Phase 1 findings summary and STOP for user confirmation before proposing changes.

Phase 2 — Canonical / masking remediation. Emit a concrete fix package: remove forwarding/masking; set real A + per-domain TXT records for BOTH apex and `www` (note: linking the apex alone does not make `www` resolve on Replit; TXT `replit-verify=` records must stay permanently); and an app-level `301` from apex+http → canonical host. Provide framework-specific redirect code (Express middleware and Next.js `redirects()`) that preserves path+query, forces HTTPS, and deliberately excludes `*.replit.app` preview and `localhost`. Include `curl -sI` verification commands with expected `301`/`200` outcomes, and a static-deploy caveat.

Phase 3 — Cannibalization audit. Pull the full post list. Cluster posts by query intent using slug + title similarity. Flag clusters where multiple URLs compete for the same intent (e.g., three "withdraw from interview" variants). For each cluster, recommend a single "keeper" URL (strongest/earliest-authority/best on-page) and generate a `301` map for the rest → keeper. Flag thin/near-duplicate volume as a domain-level quality risk that can suppress overall crawl priority.

Phase 4 — On-page AEO signals (per page). Check: answer-first opening sentence, question-shaped title matching conversational queries, valid `Article`/`FAQPage` JSON-LD, clean heading hierarchy, self-referencing canonical, internal links to related keepers, and freshness. Enforce the standing rule: every statistic must carry a named primary source — flag any unsourced stat.

Phase 5 — Authority / corroboration. Identify proprietary terms with no external corroboration and recommend anchoring each to a verifiable, named primary source so answer engines can attach the framing to a fact they already trust. Recommend the smallest set of external-citation moves that would raise citability.

Deliverables the skill produces:

- A prioritized remediation report, ordered: (1) masking/canonical fix, (2) cannibalization consolidation, (3) sitemap + request-index, (4) on-page AEO, (5) authority. Prose, not fluff; every recommendation actionable.
- A `redirect-map` file (old URL → keeper, plus apex→www).
- The framework-specific redirect code and `curl` verification block.

Bundle these helper assets and reference them from SKILL.md:

- `scripts/check_host.sh` — curls apex/www/http variants and reports 301 vs 200-with-frame (masking) and canonical-vs-link-host mismatch.

- `scripts/cluster_slugs.py` — takes a list of post URLs/titles and groups near-duplicate intents (normalize slug tokens, fuzzy-match, output clusters + suggested keeper).
- `reference/methodology.md` — the 5-phase method above as the durable playbook.

Guardrails / standing rules to encode:

- Sequence matters: forwarding/masking OFF before the app-level 301, or the redirect never runs.
- Never fabricate testimonials or stats; every stat needs a named primary source.
- Keep canonical = `www` unless the user overrides (avoids rewriting existing canonical tags).
- Read-only Phase 1 with an explicit approval gate before any change is proposed.
- Do not hardcode any single client's domain — the skill must accept any target URL.

Before writing files, restate the skill's scope and file layout for approval, then build.